Práctica 3
Analizador sintáctico de descenso predictivo

Santiago Zapata Amezcua

Compiladores 2027-1

Facultad de Ciencias, UNAM

Profesora Ariel Adara Mercado Martínez

1. Conjuntos N, Σ y símbolo inicial S

La gramática G tiene estas producciones.

programa	$\rightarrow$	declaraciones sentencias
declaraciones	$\rightarrow$	declaraciones declaracion | declaracion
declaracion	$\rightarrow$	tipo lista_var ;
tipo	$\rightarrow$	int | float
lista_var	$\rightarrow$	lista_var , identificador | identificador
sentencias	$\rightarrow$	sentencias sentencia | sentencia
sentencia	$\rightarrow$	identificador = expresion ;
	|	if (expresion) sentencias else sentencias
	|	while (expresion) sentencias
expresion	$\rightarrow$	expresion + expresion | expresion - expresion
	|	expresion * expresion | expresion / expresion
	|	identificador | numero | (expresion)

Los no terminales son los que aparecen del lado izquierdo de alguna producción.

$$N = \{\text{programa},\ \text{declaraciones},\ \text{declaracion},\ \text{tipo},$$

$$\text{lista_var},\ \text{sentencias},\ \text{sentencia},\ \text{expresion}\}$$

Los terminales son las palabras reservadas, los símbolos y los tokens identificador y numero, que son justo lo que regresa el analizador léxico.

$$\Sigma = \{\text{int},\ \text{float},\ \text{identificador},\ \text{numero},\ \text{if},\ \text{else},\ \text{while},\ ;\ ,\ ,\ ,\ =,\ (,\ ),\ +,\ -,\ *,\ /\}$$

El símbolo inicial es programa porque de ahí sale todo el programa.

$$S = \text{programa}$$

En la regla de declaracion aparece escrito lista-var, pero lo tomé como el mismo lista_var.

1.1. Eliminación de ambigüedad

La gramática tiene dos partes ambiguas.

La primera es expresion, porque no dice qué operación va primero ni cómo se agrupan. Por ejemplo, la expresión de abajo se puede armar como (a + b) * c o como a + (b * c), y las dos salen de la gramática. Con a - b - c pasa algo parecido, porque se puede agrupar por la izquierda o por la derecha.

```
a + b * c
```

Para quitarlo separé la expresión en tres niveles. expresion se encarga de la suma y la resta, termino de la multiplicación y la división, y factor de los paréntesis, los identificadores y los números. Así la multiplicación y la división siempre se hacen antes que la suma y la resta. Además cada nivel crece por la izquierda, entonces las operaciones se agrupan de izquierda a derecha.

expresion	→	expresion + termino | expresion - termino | termino
termino	→	termino * factor | termino / factor | factor
factor	→	(expresion) | identificador | numero

La segunda está en las sentencias. El cuerpo del while y lo que va después del else son de tipo sentencias, que pueden ser una o más, y como no hay llaves no se sabe dónde terminan. En el ejemplo de abajo no se sabe si b = 2 va adentro del while o después. Con lo que va después del else pasa igual.

```
while ( x ) a = 1 ; b = 2 ;
```

Para esto usé la misma idea que con el else colgante. El cuerpo se queda con todas las sentencias que siguen, hasta que aparece un else o se acaba el programa. Así cada programa se lee de una sola forma y no hay que cambiar el lenguaje, por eso las producciones de sentencia se quedan igual. Esta decisión se aplica en la tabla de análisis, como se ve en la sección 4.

Las demás producciones no son ambiguas y se quedan igual.

1.2. Eliminación de la recursividad izquierda

Una producción tiene recursividad izquierda cuando el no terminal aparece al inicio de su propio cuerpo, como en $A \to A\alpha \mid \beta$. Un analizador descendente se queda en un ciclo con eso, porque para expandir A tiene que expandir A otra vez. Se arregla cambiándola por estas dos.

$$A \to \beta A' \qquad A' \to \alpha A' \mid \varepsilon$$

Las producciones con recursividad izquierda son las de declaraciones, lista_var, sentencias, expresion y termino. Aplicando la regla quedan así.

declaraciones	$\rightarrow$	declaracion declaraciones$'$
declaraciones$'$	$\rightarrow$	declaracion declaraciones$'$ | ε
lista_var	$\rightarrow$	identificador lista_var$'$
lista_var$'$	$\rightarrow$	, identificador lista_var$'$ | ε
sentencias	$\rightarrow$	sentencia sentencias$'$
sentencias$'$	$\rightarrow$	sentencia sentencias$'$ | ε
expresion	$\rightarrow$	termino expresion$'$
expresion$'$	$\rightarrow$	+ termino expresion$'$ | - termino expresion$'$ | ε
termino	$\rightarrow$	factor termino$'$
termino$'$	$\rightarrow$	* factor termino$'$ | / factor termino$'$ | ε

Las producciones de programa, declaracion, tipo, sentencia y factor no empiezan con su propio no terminal, así que no se cambian.

1.3. Factorización izquierda

Se factoriza cuando dos producciones del mismo no terminal empiezan igual, porque el analizador no sabría cuál escoger viendo solo el siguiente token. Después de los dos pasos anteriores eso ya no pasa en ninguna producción. Las de tipo empiezan con int y con float, las de sentencia con identificador, if y while, y las de factor con paréntesis, identificador y numero. En declaraciones$'$, lista_var$'$, sentencias$'$, expresion$'$ y termino$'$ una de las opciones es ε y las demás empiezan con cosas distintas. Por eso no hace falta factorizar.

1.4. Conjuntos N' y P'

Los terminales y el símbolo inicial no cambian. Para que las tablas no queden tan anchas, de aquí en adelante escribo identificador como id.

$$N' = \{\text{programa},\ \text{declaraciones},\ \text{declaraciones}',\ \text{declaracion},\ \text{tipo},$$

$$\text{lista_var},\ \text{lista_var}',\ \text{sentencias},\ \text{sentencias}',\ \text{sentencia},$$

$$\text{expresion},\ \text{expresion}',\ \text{termino},\ \text{termino}',\ \text{factor}\}$$

Las producciones de G' quedan numeradas así. Los números son los que se usan en la tabla y en el código.

1	programa	$\rightarrow$	declaraciones sentencias
2	declaraciones	$\rightarrow$	declaracion declaraciones$'$
3	declaraciones$'$	$\rightarrow$	declaracion declaraciones$'$
4	declaraciones$'$	$\rightarrow$	ε
5	declaracion	$\rightarrow$	tipo lista_var ;
6	tipo	$\rightarrow$	int
7	tipo	$\rightarrow$	float
8	lista_var	$\rightarrow$	id lista_var$'$
9	lista_var$'$	$\rightarrow$	, id lista_var$'$
10	lista_var$'$	$\rightarrow$	ε
11	sentencias	$\rightarrow$	sentencia sentencias$'$
12	sentencias$'$	$\rightarrow$	sentencia sentencias$'$
13	sentencias$'$	$\rightarrow$	ε
14	sentencia	$\rightarrow$	id = expresion ;
15	sentencia	$\rightarrow$	if (expresion) sentencias else sentencias
16	sentencia	$\rightarrow$	while (expresion) sentencias
17	expresion	$\rightarrow$	termino expresion$'$
18	expresion$'$	$\rightarrow$	+ termino expresion$'$
19	expresion$'$	$\rightarrow$	- termino expresion$'$
20	expresion$'$	$\rightarrow$	ε
21	termino	$\rightarrow$	factor termino$'$
22	termino$'$	$\rightarrow$	* factor termino$'$
23	termino$'$	$\rightarrow$	/ factor termino$'$
24	termino$'$	$\rightarrow$	ε
25	factor	$\rightarrow$	(expresion)
26	factor	$\rightarrow$	id
27	factor	$\rightarrow$	numero

2. Conjuntos FIRST

FIRST de un no terminal son los terminales con los que puede empezar lo que sale de él. Si además puede quedar vacío, también tiene ε. Para sacarlo se revisa cada una de sus producciones. Si el cuerpo empieza con un terminal, ese terminal entra. Si empieza con un no terminal, entra su FIRST sin ε, y si ese no terminal puede quedar vacío se pasa al siguiente símbolo del cuerpo. Si la producción es ε, entra ε.

Empecé por los no terminales de abajo porque los de arriba dependen de ellos.

No terminal	Cómo se obtiene	FIRST
factor	Las producciones 25, 26 y 27 empiezan con un terminal	{ (, id, numero }
termino′	Las producciones 22 y 23 empiezan con un terminal y la 24 es ε	{ *, /, ε }
termino	Empieza con factor, que nunca es vacío	FIRST(factor) = { (, id, numero }
expresion′	Las producciones 18 y 19 empiezan con un terminal y la 20 es ε	{ +, -, ε }
expresion	Empieza con termino, que nunca es vacío	FIRST(termino) = { (, id, numero }
sentencia	Las producciones 14, 15 y 16 empiezan con un terminal	{ id, if, while }
sentencias′	La 12 empieza con sentencia y la 13 es ε	{ id, if, while, ε }
sentencias	Empieza con sentencia	{ id, if, while }
lista_var′	La 9 empieza con coma y la 10 es ε	{ , , ε }
lista_var	La 8 empieza con id	{ id }
tipo	Las producciones 6 y 7 empiezan con un terminal	{ int, float }
declaracion	Empieza con tipo	{ int, float }
declaraciones′	La 3 empieza con declaracion y la 4 es ε	{ int, float, ε }
declaraciones	Empieza con declaracion	{ int, float }
programa	Empieza con declaraciones, que nunca es vacío	{ int, float }

3. Conjuntos FOLLOW

FOLLOW de un no terminal son los terminales que pueden aparecer justo después de él. Se saca con tres reglas.

1. Al símbolo inicial se le pone $, que marca el fin de la entrada.
2. Si hay una producción $A \to \alpha B \beta$, todo FIRST(β) menos ε entra a FOLLOW(B).
3. Si B está al final de la producción o lo que le sigue puede quedar vacío, entonces todo FOLLOW(A) entra también a FOLLOW(B).

Las reglas se aplican una y otra vez hasta que ningún conjunto cambia.

No terminal	De dónde sale	FOLLOW
programa	Es el símbolo inicial	{ $ }
declaraciones	En la 1 le sigue sentencias	{ id, if, while }
declaraciones′	Está al final de la 2 y la 3, así que hereda FOLLOW(declaraciones)	{ id, if, while }
declaracion	En la 2 y la 3 le sigue declaraciones′, que puede quedar vacío, así que entra su FIRST sin ε y también FOLLOW(declaraciones)	{ int, float, id, if, while }
tipo	En la 5 le sigue lista_var	{ id }
lista_var	En la 5 le sigue el punto y coma	{ ; }
lista_var′	Está al final de la 8 y la 9, así que hereda FOLLOW(lista_var)	{ ; }
sentencias	Está al final de la 1, la 15 y la 16, así que hereda FOLLOW(programa) y FOLLOW(sentencia), y en la 15 le sigue else	{ id, if, while, else, $ }
sentencias′	Está al final de la 11 y la 12, así que hereda FOLLOW(sentencias)	{ id, if, while, else, $ }
sentencia	En la 11 y la 12 le sigue sentencias′, que puede quedar vacío, así que entra su FIRST sin ε y también FOLLOW(sentencias)	{ id, if, while, else, $ }
expresion	En la 14 le sigue el punto y coma y en la 15, 16 y 25 le sigue el paréntesis que cierra	{ ; ,) }
expresion′	Está al final de la 17, 18 y 19, así que hereda FOLLOW(expresion)	{ ; ,) }
termino	En la 17, 18 y 19 le sigue expresion′, que puede quedar vacío, así que entra su FIRST sin ε y también FOLLOW(expresion)	{ +, -, ; ,) }
termino′	Está al final de la 21, 22 y 23, así que hereda FOLLOW(termino)	{ +, -, ; ,) }
factor	En la 21, 22 y 23 le sigue termino′, que puede quedar vacío, así que entra su FIRST sin ε y también FOLLOW(termino)	{ *, /, +, -, ; ,) }

4. Tabla de análisis sintáctico predictivo

La tabla tiene un renglón por cada no terminal y una columna por cada terminal, más la columna de $. Para llenarla se toma cada producción $A \to \alpha$. Para cada terminal a que

está en FIRST(α) se pone el número de la producción en la casilla $[A, a]$. Si α puede quedar vacío, el número también va en la casilla $[A, b]$ para cada b que está en FOLLOW(A). Las casillas que quedan vacías significan error.

En esta tabla se ve en qué columnas queda cada producción y por qué.

Producción	Se usa con	Por qué
1	int, float	FIRST(declaraciones sentencias)
2	int, float	FIRST(declaracion declaraciones′)
3	int, float	FIRST(declaracion declaraciones′)
4	id, if, while	Es ε, se usa FOLLOW(declaraciones′)
5	int, float	FIRST(tipo lista_var ;)
6	int	Empieza con int
7	float	Empieza con float
8	id	Empieza con id
9	,	Empieza con coma
10	;	Es ε, se usa FOLLOW(lista_var′)
11	id, if, while	FIRST(sentencia sentencias′)
12	id, if, while	FIRST(sentencia sentencias′)
13	else, $	Es ε, se usa FOLLOW(sentencias′)
14	id	Empieza con id
15	if	Empieza con if
16	while	Empieza con while
17	(, id, numero	FIRST(termino expresion′)
18	+	Empieza con +
19	-	Empieza con -
20	; ,)	Es ε, se usa FOLLOW(expresion′)
21	(, id, numero	FIRST(factor termino′)
22	*	Empieza con *
23	/	Empieza con /
24	+, -, ; ,)	Es ε, se usa FOLLOW(termino′)
25	(	Empieza con paréntesis
26	id	Empieza con id
27	numero	Empieza con numero

Hay un solo conflicto. La producción 13 de sentencias′ es ε y en FOLLOW(sentencias′) están id, if y while, porque el cuerpo de un while o de un else puede ir seguido de otra sentencia. Entonces en las casillas [sentencias′, id], [sentencias′, if] y [sentencias′, while] caen la 12 y la 13. Es la misma ambigüedad del while y el else de la sección 1.1, y se resuelve con la misma decisión. En esas tres casillas se deja la 12, así el cuerpo se sigue quedando con las sentencias mientras vea algo que puede empezar otra. La 13 solo se usa con else y con $.

Fuera de ese caso ninguna casilla tiene dos producciones, así que con ver solo el siguiente token el analizador siempre sabe qué hacer. La tabla completa está en la siguiente página.

	int	float	id	numero	;	,	=	if	(	)	else	while	+	-	*	/	$
programa	1	1															
declaraciones	2	2															
declaraciones′	3	3	4					4				4					
declaracion	5	5															
tipo	6	7															
lista_var			8														
lista_var′					10	9											
sentencias			11					11				11					
sentencias′			12					12			13	12					13
sentencia			14					15				16					
expresion			17	17					17								
expresion′					20					20			18	19			
termino			21	21					21								
termino′					24					24			24	24	22	23	
factor			26	27					25								

Tabla 1. Tabla de análisis sintáctico predictivo de G'